

- : File Management : -

Assignment 0: Basic File and Directory Operations Tasks

1. Create a main directory named linux_practice_lab.
2. Inside it, create three directories named documents, notes, and backup.
3. Move into the documents directory and create three empty files: intro.txt, commands.txt, and practice.txt.

```
root@localhost:~# mkdir linux_practice_lab
root@localhost:~# mkdir linux_practice_lab/{documents,notes,backup}
root@localhost:~# cd documents
-bash: cd: documents: No such file or directory
root@localhost:~# cd linux_practice_lab/documents
root@localhost:~/linux_practice_lab/documents# touch intro.txt commands.txt practice.txt
```

4. Add at least five Linux commands you learned into commands.txt.
5. Display the content of commands.txt using cat, head, and tail.

```
root@localhost:~/linux_practice_lab/documents# cat commands.txt
cd
free -h
cp
mkdir
ls

root@localhost:~/linux_practice_lab/documents# head -n 5 commands.txt
cd
free -h
cp
mkdir
ls

root@localhost:~/linux_practice_lab/documents# tail -n 5 commands.txt
free -h
cp
mkdir
ls
```

6. Copy commands.txt into the backup directory.
7. Rename practice.txt to daily_practice.txt.
8. Move intro.txt from documents to notes.
9. Create a hidden file named .secret_notes inside the main directory and then list it using an option that shows hidden files.
10. Create a nested directory structure named projects/week1/day1 using a single command.

```

root@localhost:~/linux_practice_lab/documents# ls
commands.txt  daily_practice.txt
root@localhost:~/linux_practice_lab/documents# cd ..
root@localhost:~/linux_practice_lab# cd notes
root@localhost:~/linux_practice_lab/notes# ls
intro.txt
root@localhost:~/linux_practice_lab/notes# cd ..
root@localhost:~/linux_practice_lab# touch .secret_notes
root@localhost:~/linux_practice_lab# ls
backup  documents  notes
root@localhost:~/linux_practice_lab# ls -a
.  ..  backup  documents  notes  .secret_notes
root@localhost:~/linux_practice_lab# cd ..
root@localhost:~# mkdir -p projects/week1/day1
root@localhost:~# ls
anaconda-ks.cfg  linux_practice_lab  new  projects
root@localhost:~# ls -d */
linux_practice_lab/  projects/

```

11. Use both relative and absolute paths to list files from two different locations.

1. Relative path — from /root:

```
ls projects/week1/day1
```

2. Absolute path: `ls /root/linux_practice_lab/notes`

```

root@localhost:~# ls /root/linux_practice_lab/notes
intro.txt
root@localhost:~# ls projects/week1/day1
lab_observation.txt
root@localhost:~#

```

12. Remove one unnecessary test file that you create yourself, but do not remove any required assignment file.

```

root@localhost:~/projects/week1/day1# touch demo1
root@localhost:~/projects/week1/day1# ls
demo1  lab_observation.txt
root@localhost:~/projects/week1/day1# rm demo1
rm: remove regular empty file 'demo1'? y
root@localhost:~/projects/week1/day1# ls
lab_observation.txt
root@localhost:~/projects/week1/day1#

```

13. Create a file named lab_observation.txt explaining what each of these commands does: pwd, ls, cd, mkdir, touch, cp, mv, rm, and rmdir.

```
root@localhost:~/projects/week1/day1# ls
lab_observation.txt
root@localhost:~/projects/week1/day1# cat lab_observation.txt

cd -- Changes the current directory
mkdir -- Creates a new directory
touch -- Creates a new empty file
cp -- Copies files or folders
mv -- Moves or renames files/folders
rm -- Delete files
root@localhost:~/projects/week1/day1# _
```

Expert-Level Stretch Tasks

- Create the full directory structure again using fewer commands.
- Use command options to show file sizes in human-readable format.

```
root@localhost:~# cd demo1
root@localhost:~/demo1# ls
class1 class2 class3
root@localhost:~/demo1# cd class3
root@localhost:~/demo1/class3# ls
lesson1
root@localhost:~/demo1/class3# cd lesson1
root@localhost:~/demo1/class3/lesson1# ls
intro
root@localhost:~/demo1/class3/lesson1# touch intro
root@localhost:~/demo1/class3/lesson1# cat intro
linux cmds
root@localhost:~/demo1/class3/lesson1# free -h
              total        used        free      shared  buff/cache   available
Mem:           3.5Gi        478Mi        3.0Gi         8.6Mi        226Mi        3.1Gi
Swap:          975Mi           0B        975Mi
```

- Find all .txt files inside linux_practice_lab.

```
root@localhost:~/demo1/class3/lesson1# cd ../../..
root@localhost:~# cd linux_practice_lab
root@localhost:~/linux_practice_lab# find . -type f -name ".txt"
root@localhost:~/linux_practice_lab# find . -type f -name "*.txt"
./documents/commands.txt
./documents/daily_practice.txt
./notes/intro.txt
./backup/commands.txt
root@localhost:~/linux_practice_lab# _
```

- Compare the original commands.txt and the backup copy to confirm both contain the same content.

```
root@localhost:~# diff linux_practice_lab/documents/commands.txt linux_practice_lab/backup/commands.txt
root@localhost:~# cat linux_practice_lab/backup/commands.txt
cd
free -h
cp
mkdir
ls
root@localhost:~# cat linux_practice_lab/documents/commands.txt
cd
free -h
cp
mkdir
ls
root@localhost:~#
```

- Write a short note on when rm can be dangerous and how a learner should use it carefully.

Ans : The rm command is used to **delete files**, but it can be dangerous because deleted files may not be easily recoverable. A learner should always **check the file name and location before using rm**. It is better to use `rm -i`, which asks for confirmation before deleting.

Example:

```
rm -i filename.txt
```

Assignment 1: Command Stream Control and Evidence Collection

Scenario: You are working as a junior Linux administrator. Your trainer has asked you to collect system evidence from the terminal and save it into organized files without copying and pasting manually.

Learning Outcomes

- Understand the difference between standard input, standard output, and standard error.

- Use >, >>, <, 2>, and 2>&1 correctly.
- Create evidence files using commands such as whoami, id, pwd, ls, date, uname, wc, and cat.
- Separate successful command output from error output. Tasks 1. Create a directory named file_io_assignment1 and inside it create three subdirectories: reports, logs, and errors.

2. Create a file named reports/system_profile.txt using output redirection. It must contain the current date, logged-in user, user ID details, current working directory, shell name, and kernel information.

```

root@localhost:~# cd file_io_assignment1
root@localhost:~/file_io_assignment1# ls
errors  logs  reports
root@localhost:~/file_io_assignment1# date > reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat >> system_profile.txt
hello
:wg?
^C
root@localhost:~/file_io_assignment1# cat system_profile.txt
hello
:wg?
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root@localhost:~/file_io_assignment1# whoami >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
root@localhost:~/file_io_assignment1# id >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
root@localhost:~/file_io_assignment1# pwd >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
/root/file_io_assignment1
root@localhost:~/file_io_assignment1# echo $SHELL >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
/root/file_io_assignment1
/bin/bash
root@localhost:~/file_io_assignment1# uname -a >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
/root/file_io_assignment1
/bin/bash
Linux localhost.localdomain 6.12.0-55.9.1.el10_0.x86_64 #1 SMP PREEMPT_DYNAMIC Tue Mar 25 09:14:09 EDT 2025 x86_64 GNU/Linux
root@localhost:~/file_io_assignment1#

```

3. Append the output of ls -la for your home directory into reports/system_profile.txt without overwriting earlier content.

4. Run one valid command and one intentionally invalid command. Redirect normal output to logs/success.log and error output to errors/error.log.

```

root@localhost:~/file_io_assignment1# ls -la ~ >> reports/system_profile.txt
root@localhost:~/file_io_assignment1# cat reports/system_profile.txt
Fri 21 Aug 19:36:04 IST 2026
root
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
/root/file_io_assignment1
/bin/bash
Linux localhost.localdomain 6.12.0-55.9.1.el10_0.x86_64 #1 SMP PREEMPT_DYNAMIC Tue Mar 25 09:14:09 EDT 2025 x86_64 GNU/Linux
total 64
dr-xr-x---. 7 root root 4096 Aug 21 18:44 .
dr-xr-xr-x. 19 root root 249 Aug 13 22:13 ..
-rw-----. 1 root root 12288 Aug 19 23:16 .akv.txt.swp
-rw-----. 1 root root 1114 Aug 11 17:46 anaconda-ks.cfg
-rw-----. 1 root root 3130 Aug 21 19:28 .bash_history
-rw-r--r--. 1 root root 18 Feb 12 2025 .bash_logout
-rw-r--r--. 1 root root 141 Feb 12 2025 .bash_profile
-rw-r--r--. 1 root root 429 Feb 12 2025 .bashrc
-rw-r--r--. 1 root root 100 Feb 12 2025 .cshrc
drwxr-xr-x. 5 root root 48 Aug 21 16:07 demo1
drwxr-xr-x. 5 root root 73 Aug 21 19:36 file_io_assignment1
drwxr-xr-x. 5 root root 71 Aug 21 11:18 linux_practice_lab
-rw-r--r--. 1 root root 14 Aug 11 17:55 new
drwxr-xr-x. 3 root root 19 Aug 21 11:25 projects
drwx-----. 2 root root 6 Aug 11 17:40 .ssh
-rw-r--r--. 1 root root 129 Feb 12 2025 .tcshrc
-rw-----. 1 root root 14329 Aug 21 16:23 .viminfo
root@localhost:~/file_io_assignment1# mkdir -p logs errors
root@localhost:~/file_io_assignment1# ls -la > logs/success.log
root@localhost:~/file_io_assignment1# cat logs/success.log
total 8
drwxr-xr-x. 5 root root 73 Aug 21 19:36 .
dr-xr-x---. 7 root root 4096 Aug 21 18:44 ..
drwxr-xr-x. 2 root root 6 Aug 21 18:44 errors
drwxr-xr-x. 2 root root 25 Aug 21 20:32 logs
drwxr-xr-x. 2 root root 32 Aug 21 18:52 reports
-rw-r--r--. 1 root root 11 Aug 21 19:37 system_profile.txt
root@localhost:~/file_io_assignment1# ls --invalid-option 2> errors/error.log
root@localhost:~/file_io_assignment1# cat errors/error.log
ls: unrecognized option '--invalid-option'
Try 'ls --help' for more information.

```

5. Run a command that lists both an existing path and a non-existing path. Save stdout and stderr in two separate files.

```

root@localhost:~# mkdir -p logs errors
root@localhost:~# mkdir -p /tmp/testdata
root@localhost:~# echo "Application test file" > /tmp/testdata/app.txt
root@localhost:~# echo "Linux assignment test" > /tmp/testdata/linux.txt
root@localhost:~# ls -la /tmp/testdata /tmp/nothere > logs/existing_output.log 2> errors/missing_output.log
root@localhost:~# cat logs/existing_output.log
/tmp/testdata:
total 12
drwxr-xr-x. 2 root root 38 Aug 24 10:47 .
drwxrwxrwt. 12 root root 4096 Aug 24 10:44 ..
-rw-r--r--. 1 root root 22 Aug 24 10:45 app.txt
-rw-r--r--. 1 root root 22 Aug 24 10:47 linux.txt
root@localhost:~# cat errors/missing_output.log
ls: cannot access '/tmp/nothere': No such file or directory

```

6. Repeat the previous task, but combine stdout and stderr into one file named logs/combined_output.log.

```

root@localhost:~# ls -la /tmp/testdata /tmp/nothere > logs/combined_output.log 2>&1
root@localhost:~# cat logs/combined_output.log
ls: cannot access '/tmp/nothere': No such file or directory
/tmp/testdata:
total 12
drwxr-xr-x. 2 root root 38 Aug 24 18:47 .
drwxrwxrwt. 12 root root 4096 Aug 24 18:44 ..
-rw-r--r--. 1 root root 22 Aug 24 18:45 app.txt
-rw-r--r--. 1 root root 22 Aug 24 18:47 linux.txt
root@localhost:~# ls -l logs errors
errors:
total 4
-rw-r--r--. 1 root root 60 Aug 24 18:49 missing_output.log

logs:
total 12
-rw-r--r--. 1 root root 279 Aug 24 11:08 combined_output.log
-rw-r--r--. 1 root root 120 Aug 23 18:54 error.txt
-rw-r--r--. 1 root root 279 Aug 24 11:03 existing_output.log
-rw-r--r--. 1 root root 0 Aug 23 18:54 output.txt
root@localhost:~#

```

7. Create a text file named input_names.txt with at least ten names. Use input redirection with sort and save the sorted output to reports/sorted_names.txt.

```

root@localhost:~# mkdir -p reports && touch reports/{input_names.txt,sorted_names.txt}
root@localhost:~# cat > input_names.txt
Aman
Darsh
Kundan
Shubham
Suraj
Prince
Nitish
Yash
Dhoni
Rohit^C
root@localhost:~# sort < input_names.txt > reports/sorted_names.txt

```

```

bash: reports/sorted_names.txt: No such file or directory
root@localhost:~/reports# cat sorted_names.txt
Aman
Darsh
Dhoni
Kundan
Nitish
Prince
Shubham
Suraj
Yash

```

8. Use wc to count lines, words, and characters in input_names.txt and append the result to reports/system_profile.txt.

```

root@localhost:~# wc input_names.txt >> reports/system_profile.txt
root@localhost:~# cd reports
root@localhost:~/reports# cat system_profile.txt
 9  9 57 input_names.txt
root@localhost:~/reports#

```

9. Create a short reflection file named reports/reflection.txt explaining the difference between overwrite redirection and append redirection using your own words.

Ans : Overwrite redirection (>) replaces the old content of a file with new content.

Append redirection (>>) adds new content at the end of the file without deleting the old content.

Assignment 2: Log Processing Pipeline and File I/O Automation

Scenario: You are given a raw application log file. Your task is to process it using Linux command-line tools and create a clean summary report using pipelines and File I/O operations.

Starter Data

Create a file named app.log with at least 30 lines. The file must contain a mix of log levels such as INFO, WARNING, ERROR, and DEBUG. Each line should include a date, time, log level, module name, and message.

Learning Outcomes

- Use pipes to connect multiple commands.
- Filter file content using grep, sort, uniq, head, tail, and wc.
- Use tee to display output and save it at the same time.
- Generate a meaningful report from raw text files.
- Redirect unwanted output to /dev/null where appropriate.

Tasks

1. Create a directory named file_io_assignment2 with subdirectories raw, processed, reports, and archive.
2. Place app.log inside the raw directory.
3. Extract all ERROR lines from raw/app.log and save them to processed/errors_only.log.

4. Extract all WARNING lines and append them to processed/warnings_only.log.

1. Create the main directory and subdirectories

```
mkdir -p file_io_assignment2/{raw,processed,reports,archive}
```

2. Create app.log inside the raw directory

```
nano raw/app.log
```

Save : Ctrl + O → Enter → Ctrl + X

3. Extract all ERROR lines

```
grep "error" raw/app.log > processed/errors_only.log
```

4. Extract WARNING lines and append them

```
grep "WARNING" raw/app.log >> processed/warnings_only.log
```

```
grep "error" raw/app.log >> processed/errors_only.log
root@localhost:~/file_io_assignment2# grep "error" raw/app.log >> processed/errors_only.log
root@localhost:~/file_io_assignment2# cat errors_only.log
cat: errors_only.log: No such file or directory
root@localhost:~/file_io_assignment2# cd processed
root@localhost:~/file_io_assignment2/processed# cat errors_only.log
error
error nty
error
error nty
root@localhost:~/file_io_assignment2/processed# cd ..
root@localhost:~/file_io_assignment2# cd reports
root@localhost:~/file_io_assignment2/reports# cat warning_only.log
cat: warning_only.log: No such file or directory
root@localhost:~/file_io_assignment2/reports# cd ..
root@localhost:~/file_io_assignment2# cd processed
root@localhost:~/file_io_assignment2/processed# cat warning_only.log
warning
warning erg
root@localhost:~/file_io_assignment2/processed#
```

5. Create reports/log_summary.txt containing the total number of log lines, number of ERROR lines, number of WARNING lines, and number of INFO lines.

```

root@localhost:~/file_io_assignment2# echo "Total log lines : $(wc -l < raw/app.log)" > reports/log_summary.txt
root@localhost:~/file_io_assignment2# cat reports/log_summary.txt
Total log lines : 48
root@localhost:~/file_io_assignment2# echo "Error log lines : $(grep -c "error" < raw/app.log)" > reports/log_summary.txt
root@localhost:~/file_io_assignment2# cat reports/log_summary.txt
Error log lines : 2
root@localhost:~/file_io_assignment2# echo "Warning log lines : $(grep -c "warning" < raw/app.log)" > reports/log_summary.txt
root@localhost:~/file_io_assignment2# cat reports/log_summary.txt
Warning log lines : 2
root@localhost:~/file_io_assignment2# echo "Info lines : $(grep -c "info" < raw/app.log)" > reports/log_summary.txt
root@localhost:~/file_io_assignment2# cat reports/log_summary.txt
Info lines : 2
root@localhost:~/file_io_assignment2#

```

6. Build a pipeline that identifies the top three most frequently occurring log levels and save the result to reports/top_log_levels.txt.

Command : `grep -oE "INFO|WARNING|ERROR|DEBUG" raw/app.log | sort | uniq -c | sort -nr | head -3 > reports/top_log_levels.txt`

7. Use tee so that the top log levels appear on the screen and are also saved to the report file.

Command : `grep -oE "INFO|WARNING|ERROR|DEBUG" raw/app.log | sort | uniq -c | sort -nr | head -3 | tee reports/top_log_levels.txt`

```

root@localhost:~/file_io_assignment2# grep -oE "info|warning|error|debug" raw/app.log | sort | uniq -c | sort -nr | head -3 | tee reports/top_log_levels.txt
4 info
4 error
3 warning

```

8. Search for one keyword such as login, payment, network, or database and save matching lines to processed/keyword_matches.log.

9. Run a command that may produce unnecessary errors and redirect those errors to /dev/null.

```

root@localhost:~/file_io_assignment2# grep -i "login" raw/app.log > processed/keyword_matches.log
root@localhost:~/file_io_assignment2# cat processed/keyword_matches.log
login
login jj
root@localhost:~/file_io_assignment2# grep "error" raw/app.log non_existing_file 2> /dev/null
raw/app.log:error
raw/app.log:error oojo
raw/app.log:error uiou
raw/app.log:error nty

```

10. Create a final report named reports/final_observation.txt explaining what the log file indicates and which issue appears most frequently.

Create the final observation file:

Command : nano reports/final_observation.txt

Save the file: Ctrl + O → Enter → Ctrl + X

```
root@localhost:~/file_io_assignment2# cat reports/final_observation.txt
The log file contains INFO, WARNING, ERROR, and DEBUG message. The log shows some issues related to login, payment, network, and database operations.
The most frequent issue can be identified by checking the ERROR and WARNING messages. This report help us understand which problems occur more often and need at
tention.
root@localhost:~/file_io_assignment2#
```

11. Archive the complete assignment directory into a compressed file named file_io_assignment2.tar.gz.

```
root@localhost:~/file_io_assignment2# cd ..
root@localhost:~# pwd
/root
root@localhost:~# tar -czvf file_io_assignment2.tar.gz file_io_assignment2/
file_io_assignment2/
file_io_assignment2/raw/
file_io_assignment2/raw/app.log
file_io_assignment2/processed/
file_io_assignment2/processed/errors_only.log
file_io_assignment2/processed/warning_only.log
file_io_assignment2/processed/warnings_only.log
file_io_assignment2/processed/keyword_matches.log
file_io_assignment2/reports/
file_io_assignment2/reports/log_summary.txt
file_io_assignment2/reports/top_log_levels.txt
file_io_assignment2/reports/c
file_io_assignment2/reports/final_observation.txt
file_io_assignment2/archive/
file_io_assignment2/c
file_io_assignment2/reports?toplog_log_level
file_io_assignment2/log_level
file_io_assignment2/keyword_matches.log
root@localhost:~# ls -lh file_io_assignment2.tar.gz
-rw-r--r-- 1 root root 1.1K Aug 22 22:51 file_io_assignment2.tar.gz
root@localhost:~# tar -tzvf file_io_assignment2.tar.gz
drwxr-xr-x root/root      0 2026-08-22 19:29 file_io_assignment2/
drwxr-xr-x root/root      0 2026-08-22 21:04 file_io_assignment2/raw/
-rw-r--r-- root/root    326 2026-08-22 21:04 file_io_assignment2/raw/app.log
drwxr-xr-x root/root      0 2026-08-22 19:28 file_io_assignment2/processed/
-rw-r--r-- root/root     32 2026-08-22 17:46 file_io_assignment2/processed/errors_only.log
-rw-r--r-- root/root     20 2026-08-22 16:51 file_io_assignment2/processed/warning_only.log
-rw-r--r-- root/root      0 2026-08-22 17:45 file_io_assignment2/processed/warnings_only.log
-rw-r--r-- root/root     15 2026-08-22 21:48 file_io_assignment2/processed/keyword_matches.log
drwxr-xr-x root/root      0 2026-08-22 21:12 file_io_assignment2/reports/
-rw-r--r-- root/root     15 2026-08-22 18:09 file_io_assignment2/reports/log_summary.txt
-rw-r--r-- root/root     43 2026-08-22 19:04 file_io_assignment2/reports/top_log_levels.txt
-rw-r--r-- root/root      0 2026-08-22 18:46 file_io_assignment2/reports/c
-rw-r--r-- root/root    319 2026-08-22 21:48 file_io_assignment2/reports/final_observation.txt
drwxr-xr-x root/root      0 2026-08-22 16:11 file_io_assignment2/archive/
-rw-r--r-- root/root     25 2026-08-22 18:47 file_io_assignment2/c
-rw-r--r-- root/root      0 2026-08-22 18:21 file_io_assignment2/reports?toplog_log_level
-rw-r--r-- root/root      0 2026-08-22 18:22 file_io_assignment2/log_level
-rw-r--r-- root/root      0 2026-08-22 19:29 file_io_assignment2/keyword_matches.log
root@localhost:~#
```

Assignment 03: Mastering the vi Editor

Level: Beginner to Expert Foundation

Scenario: You are now comfortable creating and managing files from the Linux command line. In this assignment, you will learn how to create, edit, search, copy, delete, save, and manage text files using the vi editor.

Learning Outcomes

- Open and create files using vi.
- Understand command mode, insert mode, and last-line mode.
- Insert, edit, delete, copy, paste, undo, and redo text.
- Search and replace text inside a file.
- Save changes, quit safely, and recover from common beginner mistakes.

Tasks

1. Create a directory named vi_assignment03.
2. Inside the directory, create a file named student_profile.txt using vi.
3. Enter insert mode and write at least eight lines about yourself, your training goal, and three Linux commands you have learned.
4. Save the file and exit from vi.
5. Reopen student_profile.txt and add a heading at the first line: Linux Learning Profile.
6. Move the cursor to different lines using keyboard navigation keys and command mode movement keys.

```
"student_profile.txt" 9L, 354B written
root@localhost:~/vi_assignment03# cat student_profile.txt
Linux Learning Profile
I am learning Linux administration.
My goal is to improve my Linux and Devops skills.
I am practicing file management using the command line.
I am learning how to use the vi editor.
I have learned the ls command.
I have learned the pwd command.
I have learned the mkdir command.
I want to become confident in linux administration.
root@localhost:~/vi_assignment03#
```

7. Delete one complete line and undo the deletion.

Delete Command : dd

Undo Command : u

8. Copy one line and paste it below another line.

Copy Command : yy

Paste Command : p

9. Search for the word Linux inside the file.

Command : /Linux

```
I have learned the mkdir command.
I want to become confident in linux administration.
I want to become confident in linux administration.
```

10. Replace one word in the file using a vi search-and-replace command.

Command : :%s/Linux/UNIX/

11. Create another file named vi_commands_notes.txt and write short explanations for these vi commands: i, Esc, :w, :q, :wq, :q!, dd, yy, p, u, and /searchword.

```
root@localhost:~/vi_assignment03# cat vi_commands_notes.txt
i - Enters Insert Mode
Esc - Returns to command Mode.
:w - Saves the file.
:q - Quits vi.
:wq - Saves the file and quits vi
:q! - Quits vi without saving changes.
dd - Deletes the current line.
yy - Copies the current line.
p- Pastes the copied line below the current line.
/searchword - Searches for the specified word.
root@localhost:~/vi_assignment03#
```

12. Open both files one by one and verify their contents using cat after exiting from vi.

```
root@localhost:~/vi_assignment03# cat vi_commands_notes.txt
i - Enters Insert Mode
Esc - Returns to command Mode.
:w - Saves the file.
:q - Quits vi.
:wq - Saves the file and quits vi
:q! - Quits vi without saving changes.
dd - Deletes the current line.
yy - Copies the current line.
p- Pastes the copied line below the current line.
/searchword - Searches for the specified word.
root@localhost:~/vi_assignment03# cat student_profile.txt
Linux Learning Profile
I am learning Linux administration.
My goal is to improve my Linux and Devops skills.
I am practicing file management using the command line.
I am learning how to use the vi editor.
I have learned the ls command.
I have learned the pwd command.
I have learned the mkdir command.
I want to become confident in unix administration.

I want to become confident in unix administration.
root@localhost:~/vi_assignment03#
```

Expert-Level Stretch Tasks

- Use line numbers inside vi while editing a file.

```
1 Linux Learning Profile
2 I am learning Linux administration.
3 My goal is to improve my Linux and Devops skills.
4 I am practicing file management using the command line.
5 I am learning how to use the vi editor.
6 I have learned the ls command.
7 I have learned the pwd command.
8 I have learned the mkdir command.
9 I want to become confident in unix administration.
10
11 I want to become confident in unix administration.
12 _
```

- Jump directly to the first line, last line, and a specific line number.

Commands : first line – **gg** , last line – **G** , Specific line – e.g line no 5 (**:5**)

- Replace all occurrences of a selected word throughout the file.

Commands : **:%s/Linux/UNIX/g**

% → Entire file

s → Substitute

Linux → Old word

UNIX → New word

g → All occurrences

- Copy three consecutive lines and paste them at the end of the file.

Commands : **:3,3yy, G, p**

- Create a small troubleshooting note explaining what to do if you enter the wrong mode or cannot exit from vi.

vi Troubleshooting Notes :

If I cannot type text:

Press Esc and then press i to enter Insert Mode.

If I enter the wrong mode:

Press Esc to return to Command Mode.

If I want to save:

Press Esc, type :w, and press Enter.

If I want to save and exit:

Press Esc, type :wq, and press Enter.

If I want to exit without saving:

Press Esc, type :q!, and press Enter.

If I cannot exit from vi:

Press Esc first, then use :q! to exit without saving.

If I am confused about the current mode:

Press Esc. This returns to Command Mode.

```
root@localhost:~/vi_assignment03# cat vi_troubleshooting.txt
```

If I cannot exit from vi:

Press Esc first, then use :q! to exit without saving and :wq! and enter to exit after saving.

```
root@localhost:~/vi_assignment03#
```